

Exercise: DDL, Constraints & Inserts, Updates, Deletes

I made the SQL code **purple text on this document**

Part A

A1. Missing constraints on devices

The devices table is missing these constraints:

Column	Missing Constraint	Why
device_id	PRIMARY KEY	Each device needs a unique identifier and cannot be NULL.
asset_tag	NOT NULL	Every tracked device should have an asset tag.
asset_tag	UNIQUE	Two devices should not share the same asset tag.
device_type	NOT NULL	FleetDesk needs to know what kind of device it is.
location	NOT NULL	The device's location should be recorded.
status	NOT NULL	Every device needs a defined lifecycle status.
status	CHECK	Status must be one of: active, repair, retired, or lost.

-device_id is already an identity column, which automatically generates values, but identity does not make it a primary key. The primary key still needs to be explicitly declared.

A2.

```
ALTER TABLE devices
ADD CONSTRAINT pk_devices
PRIMARY KEY (device_id);
```

```
ALTER TABLE devices
ALTER COLUMN asset_tag SET NOT NULL;
```

```
ALTER TABLE devices
ADD CONSTRAINT uq_devices_asset_tag
UNIQUE (asset_tag);
```

```
ALTER TABLE devices
ALTER COLUMN device_type SET NOT NULL;
```

```
ALTER TABLE devices
ALTER COLUMN location SET NOT NULL;
```

```
ALTER TABLE devices
ALTER COLUMN status SET NOT NULL;
```

```
ALTER TABLE devices
ADD CONSTRAINT chk_devices_status
CHECK (status IN ('active', 'repair', 'retired', 'lost'));
```

What these accomplish:

Primary key – prevents duplicate or NULL device_id values.

NOT NULL – prevents incomplete device records.

UNIQUE – prevents two devices from having the same asset tag.

CHECK – prevents invalid statuses such as 'broken', 'unknown', or 'deleted'.

A3. Fully constrain alerts

For alerts, we need:

1. Primary key on alert_id
2. Foreign key from device_id – devices(device_id)
3. Foreign key from department_id – departments(department_id)
4. device_id should be NOT NULL
5. department_id can reasonably remain nullable
6. severity needs a CHECK constraint
7. message needs NOT NULL
8. occurred_at needs a default

The important distinction is that an alert must identify the device that generated it, but a department may not always be known or applicable when the alert is created.

```
ALTER TABLE alerts
```

```
ADD CONSTRAINT pk_alerts
```

```
PRIMARY KEY (alert_id);
```

```
ALTER TABLE alerts
```

```
ALTER COLUMN device_id SET NOT NULL;
```

```
ALTER TABLE alerts
```

```
ADD CONSTRAINT fk_alerts_device
```

```
FOREIGN KEY (device_id)
```

```
REFERENCES devices(device_id);
```

```
ALTER TABLE alerts
```

```
ADD CONSTRAINT fk_alerts_department
```

```
FOREIGN KEY (department_id)
```

```
REFERENCES departments(department_id);
```

```
ALTER TABLE alerts
```

```
ADD CONSTRAINT chk_alerts_severity
```

```
CHECK (severity IN ('info', 'warning', 'error', 'critical'));
```

```
ALTER TABLE alerts
```

```
ALTER COLUMN message SET NOT NULL;
```

```
ALTER TABLE alerts
```

```
ALTER COLUMN occurred_at SET DEFAULT CURRENT_TIMESTAMP;
```

Why make device_id NOT NULL but leave department_id nullable?

An alert is specifically defined as an automated monitoring alert against a device, so an alert without a device would be invalid FleetDesk data. The department may be unknown, unavailable, or not directly associated with the device at the time the alert is generated, so allowing department_id to be NULL is more defensible.

Part B — The Nullable-FK Judgment

B1. work_orders.assignee_technician_id

NULL is a legitimate value because a work order can exist before a technician has been assigned to it. In this case, NULL correctly represents “unassigned”, rather than requiring FleetDesk to create a fake technician or assign the work order to someone who is not actually responsible for it.

B2. Example: work_orders.kind

kind should be NOT NULL because every work order must identify what type of request it is, such as an incident or change request. A NULL value would leave FleetDesk unable to determine what kind of work order it is handling, so an empty value represents incomplete/invalid data rather than a legitimate state.

Part C - The Cascade Tradeoff

C1. alerts – devices

I would choose ON DELETE CASCADE for the foreign key from alerts to devices. Alerts are dependent records owned by the device they monitor, so when the device is permanently removed from inventory, its associated alerts can also be removed; this passes the ownership test because the alerts have no meaningful existence in FleetDesk without their parent device.

```
ALTER TABLE alerts
```

```
DROP CONSTRAINT fk_alerts_device;
```

```
ALTER TABLE alerts
```

```
ADD CONSTRAINT fk_alerts_device
```

```
FOREIGN KEY (device_id)
```

```
REFERENCES devices(device_id)
```

```
ON DELETE CASCADE;
```

C2. work_orders – technicians

I would choose ON DELETE SET NULL for work_orders.assigned_technician_id. A work order belongs to FleetDesk, not to the technician assigned to it, so deleting a technician should not delete the work order; instead, the assignment becomes NULL, correctly representing that the work order is now unassigned.

```
ALTER TABLE work_orders
```

```
ADD CONSTRAINT fk_work_orders_technician
```

```
FOREIGN KEY (assigned_technician_id)
```

```
REFERENCES technicians(technician_id)
```

```
ON DELETE SET NULL;
```

Key distinction:

-Alerts – devices: CASCADE because the alert is dependent on its device.

-Work orders – technicians: SET NULL because the work order survives when its assigned technician leaves.

Part D — A Transactional Change

D1.

BEGIN;

ALTER TABLE work_orders

ADD COLUMN sla_breached BOOLEAN NOT NULL DEFAULT FALSE;

UPDATE work_orders

SET sla_breached = TRUE

WHERE status = 'open'

AND priority = 'P1'

AND opened_at < TIMESTAMP '2026-01-01 00:00:00';

CREATE INDEX idx_work_orders_sla_breached

ON work_orders (sla_breached);

COMMIT;

D2. If the UPDATE statement fails, a ROLLBACK (instead of COMMIT) removes everything performed inside that transaction, including the new column, any successful updates, and the index creation. Operationally, this matters because the database returns to its original consistent state instead of leaving behind a partially completed schema/data change that could cause the application and dashboard to disagree about the rollout.

Views Part E —

E1. Create View:

```
CREATE VIEW open_p1_work_orders_by_department AS

SELECT

    d.name AS department_name,

    COUNT(w.work_order_id) AS open_p1_count

FROM departments d

JOIN work_orders w

    ON w.department_id = d.department_id

WHERE w.status = 'open'

    AND w.priority = 'P1'

GROUP BY d.department_id, d.name;
```

Now all three dashboards and the email report can simply use:

```
SELECT *

FROM open_p1_work_orders_by_department;
```

E2. This should stay a saved query, not become a view. The analysis is a one-off historical audit that will not be reused, so creating a persistent database object adds unnecessary maintenance and schema overhead; a view is more appropriate when the result is reused, standardized, or consumed by multiple users/applications.

Part F – Spot the Unbounded Write

F1. The statement updates every row in the work_orders table, changing the status of every work order to 'resolved'. Because there is no WHERE clause, the database has no way to know that only work order 4521 was intended. This is an unbounded UPDATE and could cause major data corruption in production.

F2.

```
UPDATE work_orders

SET status = 'resolved'
```

```
WHERE work_order_id = 4521;
```

Before running it in production, I would first run the same WHERE condition as a SELECT to verify that it returns exactly the intended ticket, then review the UPDATE for a restrictive WHERE clause and, when appropriate, execute it inside a transaction so the change can be rolled back if something is wrong.

Part G — Reflection

I would choose the CHECK constraint on devices.status. Without it, a bulk-import script could insert a device with a status such as 'broken', 'unknown', or 'deleted', even though FleetDesk only recognizes active, repair, retired, and lost. The database constraint CHECK (status IN ('active', 'repair', 'retired', 'lost')) would evaluate the attempted value and reject the entire write because 'broken' is not in the approved set. This protects the database even when a new integration or script bypasses the application's front-end validation.

Part H — Basic DML From Scratch

H1. Insert the new laptop:

```
INSERT INTO devices (asset_tag, device_type, location, status)

VALUES ('AT-2044', 'laptop', 'Library 2F', 'active');
```

H2. Mark technician jmiller inactive:

```
UPDATE technicians

SET active = FALSE

WHERE username = 'jmiller';
```

H3. Delete exactly the duplicate dept.:

```
DELETE FROM departments

WHERE name = 'IT Support (temp)'

AND building = 'Annex';
```